

Hospital Management System

Complete Project Documentation

Java Desktop Application with MySQL

Date: Tue 14 Jul 2026

Table of Contents

- 1. Project Overview
- 2. System Architecture
- 3. Database Design
- 4. Class Structure and Description
- 5. User Interface Modules
- 6. Business Logic and Validation
- 7. Features and Functionality
- 8. Technical Specifications
- 9. Installation and Setup
- 10. User Guide
- Appendix A: Sample Data
- Appendix B: Code Snippets
- Appendix C: Testing
- Conclusion

1. Project Overview

1.1 Introduction

The Hospital Management System (HMS) is a comprehensive Java-based desktop application designed to streamline hospital operations. The system provides a centralized platform for managing patients, doctors, appointments, and billing processes.

1.2 Objectives

- Efficient Patient Management: Maintain complete patient records
- Doctor Management: Manage doctor profiles and specializations
- Appointment Scheduling: Handle appointment booking and tracking
- Billing System: Generate and manage patient bills
- Report Generation: Create patient and billing reports
- Data Persistence: Store all data in MySQL database

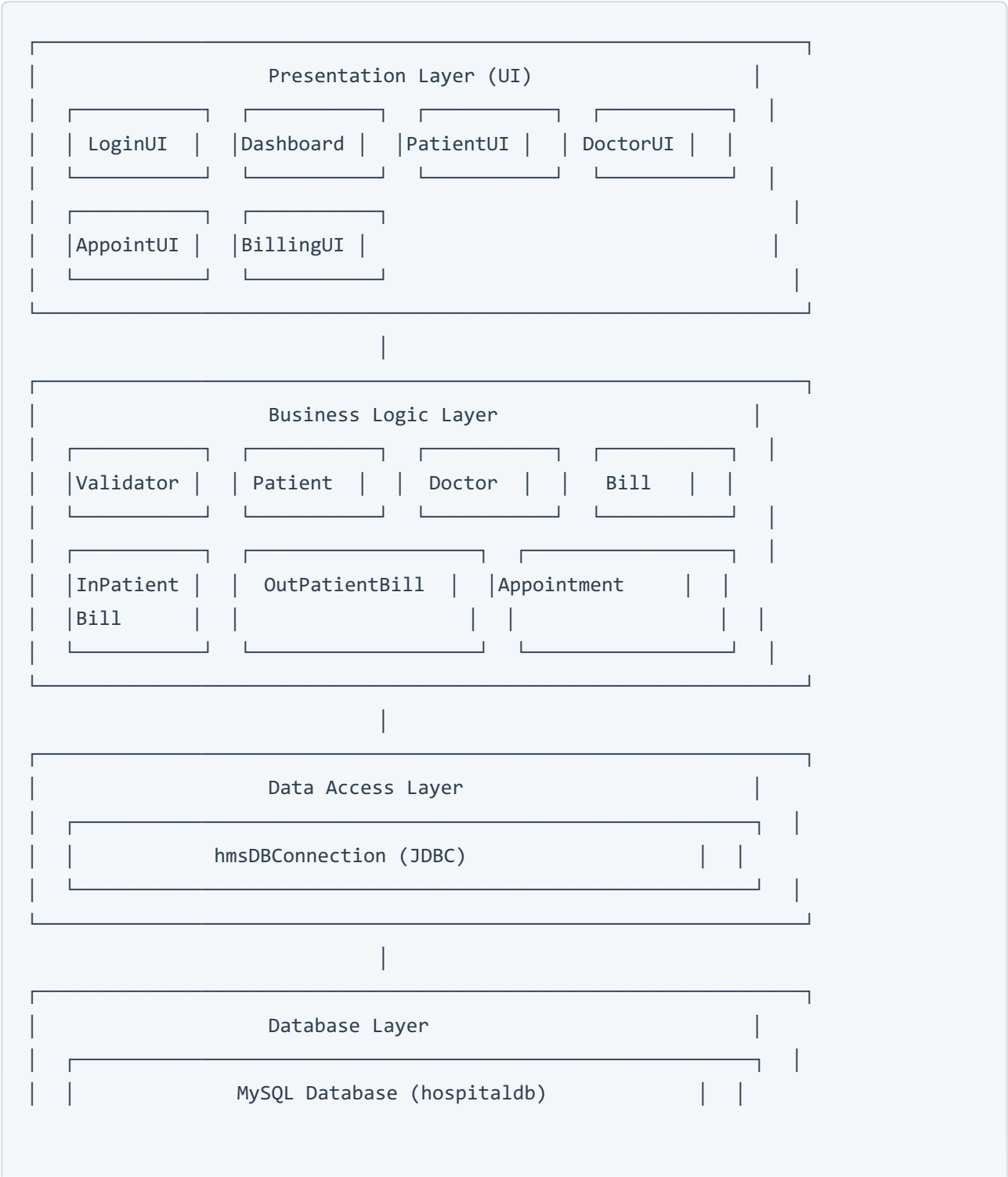
1.3 Key Features

- User authentication with login system
- CRUD operations for patients, doctors, and appointments
- Dynamic billing with in-patient and out-patient support
- Automated backup system
- Report generation capabilities
- Intuitive GUI interface using Swing

2. System Architecture

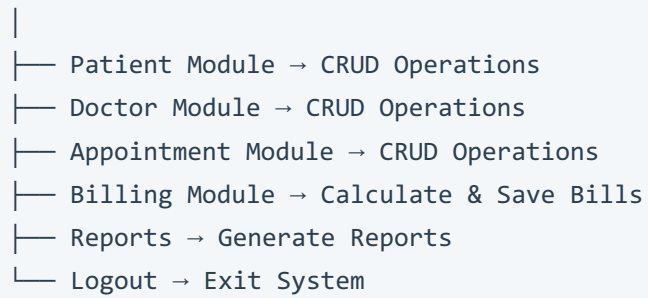
2.1 Architecture Overview

The system follows a Model-View-Controller (MVC) architecture pattern:



2.2 System Flow Diagram

User Login → Dashboard → Module Selection



3. Database Design

3.1 Database Schema

Database Name: hospitaldb

Table 1: users

```
CREATE TABLE users (  
    username VARCHAR(50) PRIMARY KEY,  
    password VARCHAR(50) NOT NULL  
);
```

Table 2: patients

```
CREATE TABLE patients (  
    patientid INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    age INT NOT NULL,  
    gender VARCHAR(10),  
    disease VARCHAR(100),  
    mobile BIGINT,  
    address VARCHAR(200)  
);
```

Table 3: doctors

```
CREATE TABLE doctors (  
    doctorid INT PRIMARY KEY,  
    doctorname VARCHAR(100) NOT NULL,  
    specialization VARCHAR(50),  
    fees DECIMAL(10,2),  
    mobile BIGINT  
);
```

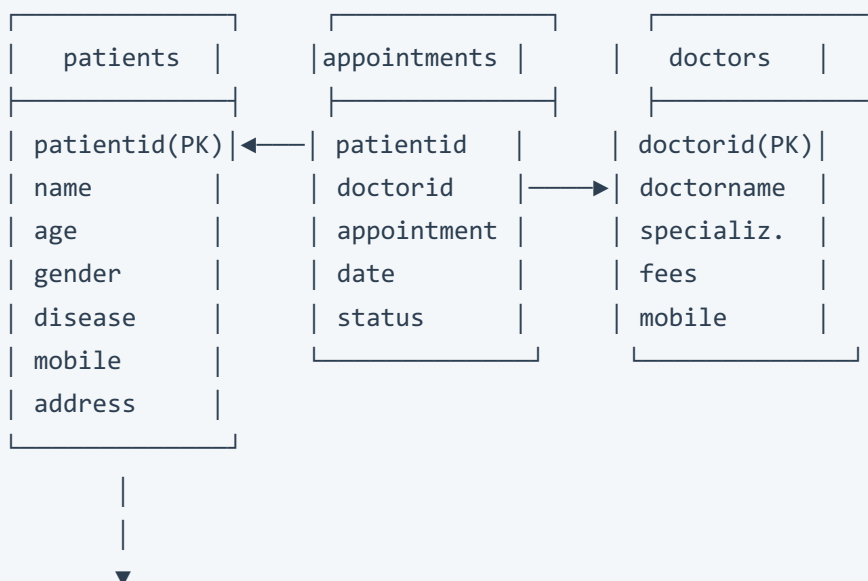
Table 4: appointments

```
CREATE TABLE appointments (  
    appointmentid INT PRIMARY KEY,  
    patientid INT,  
    doctorid INT,  
    appointmentdate DATE,  
    status VARCHAR(20),  
    FOREIGN KEY (patientid) REFERENCES patients(patientid),  
    FOREIGN KEY (doctorid) REFERENCES doctors(doctorid)  
);
```

Table 5: bills

```
CREATE TABLE bills (  
    billid INT PRIMARY KEY,  
    patientid INT,  
    consultationfees DECIMAL(10,2),  
    medicinecharges DECIMAL(10,2),  
    roomcharges DECIMAL(10,2),  
    totalamount DECIMAL(10,2),  
    FOREIGN KEY (patientid) REFERENCES patients(patientid)  
);
```

3.2 Entity Relationship Diagram



bills
billid(PK)
patientid
consultation
fees
medicine
charges
roomcharges
totalamount

4. Class Structure and Description

4.1 Entity Classes (Models)

Patient Class

```
package hospitalmng;

public class Patient {
    private int patientId;
    private String name;
    private int age;
    private String gender;
    private String disease;
    private long mobile;
    private String address;

    // Constructor with all fields
    // Getters for all fields
}
```

Purpose: Represents patient entity with all attributes and getter methods.

Doctor Class

```
package hospitalmng;

public class Doctor {
    private int doctorId;
    private String doctorName;
    private String specialization;
    private double fees;
    private long mobile;

    // Constructor and getters
}
```

Purpose: Represents doctor entity with professional details.

Appointment Class

```
package hospitalmng;

import java.sql.Date;

public class Appointment {
    private int appointmentId;
    private int patientId;
    private int doctorId;
    private Date appointmentDate;
    private String status;

    // Constructor and getters
}
```

Purpose: Represents appointment entity linking patients and doctors.

Bill Class (Base Class)

```
package hospitalmng;

public class Bill {
    protected int billId;
    protected int patientId;
    protected double consultationFees;
    protected double medicineCharges;

    public double calculateTotal() {
        return consultationFees + medicineCharges;
    }
}
```

Purpose: Base class for billing with basic attributes and calculation.

InPatientBill Class (Subclass)

```
package hospitalmng;
```

```
public class InPatientBill extends Bill {  
    private double roomCharges;  
  
    @Override  
    public double calculateTotal() {  
        return consultationFees + medicineCharges + roomCharges;  
    }  
}
```

Purpose: Extends Bill to include room charges for in-patients.

OutPatientBill Class (Subclass)

```
package hospitalmng;  
  
public class OutPatientBill extends Bill {  
    // Reuses parent class calculation  
}
```

Purpose: Extends Bill for out-patient billing (no room charges).

4.2 User Interface Classes (Views)

LoginUI

Purpose: User authentication interface

Components: Username field, Password field, Login/Reset buttons

Functionality: Validates credentials against database

Dashboard

Purpose: Main navigation hub

Components: Navigation buttons for all modules

Functionality: Opens respective module windows, generates reports, starts auto-backup thread, handles logout.

PatientUI

Purpose: Patient management interface

Components: Input fields, CRUD buttons, Table display

Functionality: Add, Update, Delete, Search, Display all patients

DoctorUI

Purpose: Doctor management interface

Components: Input fields, Specialization dropdown, CRUD buttons

Functionality: Complete CRUD operations with validation

AppointmentUI

Purpose: Appointment management interface

Components: Date input, Status dropdown, CRUD buttons

Functionality: Schedule, update, cancel appointments

BillingUI

Purpose: Billing management interface

Components: Patient type dropdown, Calculation, Bill saving

Functionality: Calculate total bill based on patient type, save bills to database, display all bills.

4.3 Utility Classes

hmsDBConnection

```
package hospitalmng;

import java.sql.Connection;
import java.sql.DriverManager;

public class hmsDBConnection {
    static Connection con;

    public static Connection getConnection() {
        // Database connection setup
        // Returns Connection object
    }
}
```

Purpose: Singleton database connection manager.

Validator

```
package hospitalmng;

public class Validator {
    public static void validatePatient(String name, int age, long mobile)
        throws InvalidDataException;

    public static void validateDoctorFees(double fees)
        throws InvalidDataException;
}
```

Purpose: Data validation utilities with custom exception handling.

InvalidDataException

```
package hospitalmng;

public class InvalidDataException extends Exception {
```

```
    public InvalidDataException(String message) {  
        super(message);  
    }  
}
```

Purpose: Custom exception for validation errors.

ReportGenerator

Purpose: Generates text file reports from database data.

AutoBackup

```
package hospitalmng;  
  
public class AutoBackup implements Runnable {  
    @Override  
    public void run() {  
        // Automatic report generation every 10 seconds  
    }  
}
```

Purpose: Background thread for automatic report generation.

PatientCollection

```
package hospitalmng;  
  
import java.util.ArrayList;  
import java.util.HashMap;  
  
public class PatientCollection {  
    ArrayList<Patient> patientList;  
    HashMap<Integer, Patient> patientMap;  
  
    public void addPatient(Patient p);  
    public void displayPatients();  
    public void searchPatient(int id);  
}
```

Purpose: Demonstrates collection framework usage (ArrayList and HashMap).

4.4 Test Classes

- **TestCollection:** Tests PatientCollection functionality; demonstrates adding and searching patients.
- **TestConnection:** Tests database connectivity; verifies JDBC connection.
- **TestReport:** Tests report generation functionality.

5. User Interface Modules

5.1 Login Screen

HOSPITAL MANAGEMENT SYSTEM

Username: [_____]

Password: [_____]

[Login]

[Reset]

Purpose: Secure system access.

Validation: Checks credentials against users table.

Success Action: Opens Dashboard.

5.2 Dashboard

HOSPITAL MANAGEMENT SYSTEM

Patient Module

Doctor Module

Appointment Module

Billing Module

Logout

Generate Patient Report

Generate Billing Report

Purpose: Central navigation hub.

5.3 Patient Module UI

Patient Management

Patient ID: [] Name: []
Age: [] Gender: []
Disease: [] Mobile: []
Address: []

[Add] [Update] [Delete] [Search] [Display All] [Clear]

ID	Name	Age	Gender	Disease	Mobile	Addr
101	Ravi	25	Male	Fever	98765	...
102	Suresh	30	Male	Diab...	91234	...

5.4 Doctor Module UI

Doctor Management																				
Doctor ID: [_____] Doctor Name: [_____]																				
Specialization: [▼ Cardiologist]																				
Consultation Fees: [_____] Mobile: [_____]																				
[Add] [Update] [Delete] [Search] [Display All] [Clear]																				
<table><thead><tr><th>ID</th><th>Doctor Name</th><th>Specialization</th><th>Fees</th><th>Mobile</th></tr></thead><tbody><tr><td>201</td><td>Dr. Sharma</td><td>Cardiologist</td><td>1000</td><td>98765</td></tr><tr><td>202</td><td>Dr. Reddy</td><td>Neurologist</td><td>1200</td><td>91234</td></tr></tbody></table>						ID	Doctor Name	Specialization	Fees	Mobile	201	Dr. Sharma	Cardiologist	1000	98765	202	Dr. Reddy	Neurologist	1200	91234
ID	Doctor Name	Specialization	Fees	Mobile																
201	Dr. Sharma	Cardiologist	1000	98765																
202	Dr. Reddy	Neurologist	1200	91234																

5.5 Appointment Module UI

Appointment Management																				
Appoint ID: [_____] Patient ID: [_____]																				
Doctor ID: [_____] Date: [yyyy-mm-dd]																				
Status: [▼ Booked]																				
[Add] [Update] [Delete] [Search] [Display All] [Clear]																				
<table><thead><tr><th>App ID</th><th>Patient</th><th>Doctor</th><th>Date</th><th>Status</th></tr></thead><tbody><tr><td>301</td><td>101</td><td>201</td><td>2024-01-15</td><td>Booked</td></tr><tr><td>302</td><td>102</td><td>202</td><td>2024-01-16</td><td>Completed</td></tr></tbody></table>						App ID	Patient	Doctor	Date	Status	301	101	201	2024-01-15	Booked	302	102	202	2024-01-16	Completed
App ID	Patient	Doctor	Date	Status																
301	101	201	2024-01-15	Booked																
302	102	202	2024-01-16	Completed																

5.6 Billing Module UI

Billing System

Bill ID: [_____] Patient ID: [_____] Consultation Fees: [_____] Medicine Charges: [_____] Room Charges: [_____] Patient Type: [▼ InPatient]

[Save Bill] [Calculate] [Display Bills] [Clear]

Bill	Patient	Consult	Medicine	Room	Total	
401	101	500.00	200.00	0.00	700.00	
402	102	500.00	300.00	500.00	1300.0	

6. Business Logic and Validation

6.1 Validation Rules

Patient Validation

```
public static void validatePatient(String name, int age, long mobile)
    throws InvalidDataException {

    // Rule 1: Name cannot be empty
    if(name.trim().isEmpty()) {
        throw new InvalidDataException("Name cannot be empty");
    }

    // Rule 2: Age must be positive
    if(age <= 0) {
        throw new InvalidDataException("Invalid age");
    }

    // Rule 3: Mobile number must be exactly 10 digits
    String mob = String.valueOf(mobile);
    if(mob.length() != 10) {
        throw new InvalidDataException("Mobile number must be 10 digits");
    }
}
```

Doctor Validation

```
public static void validateDoctorFees(double fees)
    throws InvalidDataException {

    // Rule: Consultation fees must be greater than zero
    if(fees <= 0) {
        throw new InvalidDataException("Fees must be greater than zero");
    }
}
```

6.2 Business Rules

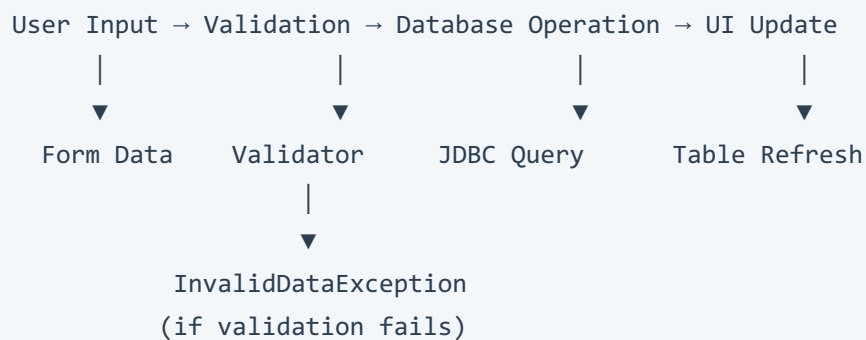
Billing Logic:

- Out-Patient Bill: Total = Consultation Fees + Medicine Charges
- In-Patient Bill: Total = Consultation Fees + Medicine Charges + Room Charges

Appointment Status:

- Booked (Initial state)
- Completed (After consultation)
- Cancelled (If cancelled)

6.3 Data Flow



7. Features and Functionality

7.1 CRUD Operations

Module	Create	Read	Update	Delete
Patient	✓	✓	✓	✓
Doctor	✓	✓	✓	✓
Appointment	✓	✓	✓	✓
Bill	✓	✓	✗	✗

7.2 Search Functionality

- Patient: Search by Patient ID
- Doctor: Search by Doctor ID
- Appointment: Search by Appointment ID

7.3 Display All

- Shows all records in tabular format
- Auto-refresh table
- Clear button clears form and table

7.4 Report Generation

Patient Report (patients_report.txt)

```
===== PATIENT REPORT =====
```

```
Patient ID : 101
```

```
Name : Ravi
```

```
Age : 25
```

```
Gender : Male
```

```
Disease : Fever
```

```
Mobile : 9876543210
```

```
Address : Vijayawada
```

```
-----
```

Billing Report (billing_report.txt)

```
===== BILLING REPORT =====
```

```
Bill ID : 401
```

```
Patient ID : 101
```

```
Consultation Fees : 500.0
```

```
Medicine Charges : 200.0
```

```
Room Charges : 0.0
```

```
Total Amount : 700.0
```

```
-----
```

7.5 Auto-Backup System

- **Type:** Background thread
- **Frequency:** Every 10 seconds
- **Function:** Automatically generates reports
- **Implementation:** Runnable interface

7.6 Collection Framework Demonstration

PatientCollection Class

- ArrayList: Maintains patient list (ordered)
- HashMap: Provides fast ID-based lookup

TestCollection Output

```
101 Ravi Fever  
102 Suresh Diabetes  
Patient Found  
101 Ravi Fever
```

8. Technical Specifications

8.1 Technology Stack

Component	Technology
Language	Java (JDK 8+)
Database	MySQL 8.0
JDBC Driver	MySQL Connector/J 8.0
GUI Framework	Swing
IDE	Eclipse / IntelliJ / NetBeans

8.2 System Requirements

Minimum Requirements:

- Operating System: Windows 7+ / Linux / macOS
- RAM: 4 GB
- Storage: 500 MB free space
- Database: MySQL 8.0 installed
- Java: JDK 8 or higher

8.3 Dependencies

MySQL Connector (pom.xml or manual)

```
<dependency>  
  <groupId>mysql</groupId>  
  <artifactId>mysql-connector-java</artifactId>
```

```
<version>8.0.33</version>  
</dependency>
```

8.4 Package Structure

```
hospitalmng/  
├─ Appointment.java  
├─ AppointmentUI.java  
├─ AutoBackup.java  
├─ Bill.java  
├─ BillingUI.java  
├─ Dashboard.java  
├─ Doctor.java  
├─ DoctorUI.java  
├─ hmsDBConnection.java  
├─ InPatientBill.java  
├─ InvalidDataException.java  
├─ LoginUI.java  
├─ OutPatientBill.java  
├─ Patient.java  
├─ PatientCollection.java  
├─ PatientUI.java  
├─ ReportGenerator.java  
├─ TestCollection.java  
├─ testconnection.java  
├─ TestReport.java  
└─ Validator.java
```

9. Installation and Setup

9.1 Database Setup

Step 1: Create Database

```
CREATE DATABASE hospitaldb;  
USE hospitaldb;
```

Step 2: Create Tables

```
-- Users table  
CREATE TABLE users (  
    username VARCHAR(50) PRIMARY KEY,  
    password VARCHAR(50) NOT NULL  
);  
  
-- Insert default user  
INSERT INTO users VALUES('admin', 'admin123');  
  
-- Patients table  
CREATE TABLE patients (  
    patientid INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    age INT NOT NULL,  
    gender VARCHAR(10),  
    disease VARCHAR(100),  
    mobile BIGINT,  
    address VARCHAR(200)  
);  
  
-- Doctors table  
CREATE TABLE doctors (  
    doctorid INT PRIMARY KEY,  
    doctorname VARCHAR(100) NOT NULL,  
    specialization VARCHAR(50),  
    fees DECIMAL(10,2),  
    mobile BIGINT
```

```
);
```

```
-- [Followed by appointments and bills tables as defined in Section 3]
```

9.2 Project Setup

Step 1: Configure Database Connection

Update hmsDBConnection.java with your credentials:

```
con = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/hospitaldb",  
    "root",                               // Your username  
    "Mahi@1258"                           // Your password  
);
```

Step 2: Add MySQL Connector

- Download mysql-connector-java-8.0.33.jar
- Add to classpath in your IDE.

Step 3: Compile and Run

```
javac -cp ".;mysql-connector-java-8.0.33.jar" hospitalmng/*.java  
java -cp ".;mysql-connector-java-8.0.33.jar" hospitalmng.LoginUI
```

10. User Guide

10.1 Getting Started

Launch application, Login with 'admin'/'admin123'. System opens Dashboard and background backup thread starts automatically.

10.2 Module Operations

Patient Module: Enter details, click Add. Use Search/Update/Delete for record management.

Doctor Module: Manage specializations and fees. CRUD operations same as Patient.

Appointment Module: Reference existing Patient/Doctor IDs. Set dates in yyyy-mm-dd format.

Billing Module: Select In-Patient or Out-Patient. Click Calculate, then Save Bill.

Appendix A: Sample Data

```
INSERT INTO patients VALUES
(101, 'Ravi', 25, 'Male', 'Fever', 9876543210, 'Vijayawada'),
(102, 'Suresh', 30, 'Male', 'Diabetes', 9123456780, 'Hyderabad');

INSERT INTO doctors VALUES
(201, 'Dr. Sharma', 'Cardiologist', 1000.00, 9876543211);
```

Appendix B: Code Snippets

Database Connection

```
public class hmsDBConnection {
    static Connection con;
    public static Connection getConnection() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/hospitaldb", "root", "Mahi@1258");
        } catch (Exception e) { e.printStackTrace(); }
        return con;
    }
}
```

Conclusion

The Hospital Management System successfully demonstrates a complete CRUD-based desktop application using Java and MySQL. The modular architecture makes it easy to extend with pharmacy or inventory features.